¡Ya casi empezamos!



Centro de e-Learning

UTN.BA

Clase 9

Persistencia e integración
Backend-as-a-Service

Centro de e-Learning
UTN.BA

¿Dudas con el Contenido previo?

Centro de e-Learning

UTN.BA

Objetivos de la Clase

- **Aprender** a trabajar con almacenamiento local (AsyncStorage) y remoto (Firebase, AWS) para crear apps que mantengan datos entre sesiones.
- **Manipular** la sincronización en la nube.

¿Qué es la persistencia de datos?

La persistencia permite que los datos sobrevivan a cierres de sesión, reinicios o incluso reinstalaciones de la app. Es fundamental para experiencias personalizadas, sesiones activas, historial de acciones o contenido offline.

¿Por qué es importante?

- Conserva sesiones y preferencias del usuario
- Permite funcionamiento en modo offline
- Mejora la velocidad al evitar llamadas redundantes
- Facilita sincronización con bases de datos remotas

Tipos de Persistencia de Datos

Tipo	Alcance	Ejemplo
Memoria	Volátil, se pierde al cerrar app	`useState`, `useContext`
Local	Persistente, dispositivo	`AsyncStorage`, SQLite
Remota	Persistente, servidor o nube	Firebase, AWS, MongoDB Atlas

¿Qué es AsyncStorage?

Es un sistema de almacenamiento clave-valor local y persistente. Usa APIs nativas bajo el capó (UserDefaults, SharedPreferences) pero con una interfaz JS sencilla.

Ideal para:

- Tokens de sesión
- Idioma, preferencias
- Datos temporales para sincronizar luego

Este no es el único formato Local

Además de AsyncStorage podemos encontrar otros formatos locales de almacenar información, siendo SQLite el competidor más común. Aún así también existen otros menos comunes que se utilizan menos debido a su uso más complejo o gran similitud con los ya mencionados:

- **MMKV** (librería de Meta)
- **SecureStore** (librería de expo, más común para encriptar)
- **FileSystem** (anteriormente común, hoy en día en desuso ya que puede ocupar mucho espacio del dispositivo)

Instalación y uso básico de AsyncStorage

Instalación

```
npx expo install @react-native-async-storage/async-storage
```

Importación

```
import AsyncStorage from
'@react-native-async-storage/async-storage';
```

Operación CRUD en AsyncStorage:

Guardar:

```
await AsyncStorage.setItem('usuario', JSON.stringify({ nombre: 'Juan' }));
```

Leer:

```
const data = await AsyncStorage.getItem('usuario');
console.log(JSON.parse(data));
```

Eliminar:

```
await AsyncStorage.removeItem('usuario');
```

Limpiar todo:

```
await AsyncStorage.clear();
```

Guardar múltiples valores:

```
await AsyncStorage.multiSet([['tema', 'oscuro'], ['idioma', 'es']]);
```

Buenas prácticas con AsyncStorage

- **Usar try/catch en todas las llamadas**
  - De no hacerse, podría crashear el programa o mostrarse errores de código directamente al usuario.
- **Serializar objetos complejos con JSON.stringify()**
  - De no hacerse, podrían haber errores de Parsing.
- **Evitar claves genéricas. Usá prefijos: @app:usuario**
  - Utilizar claves genéricas te hace más vulnerable a usuarios maliciosos.
- **Nunca guardes contraseñas sin cifrado**
  - No utilizar cifrado permite el acceso a usuarios maliciosos a información del resto de tus usuarios.

¿Qué es Firebase?

Es una plataforma de Google para crear backends completos sin necesidad de configurar servidores.

Incluye:

- Autenticación
- Firestore (base de datos en tiempo real)
- Almacenamiento
- Notificaciones

Ventajas clave de Firebase

- **Escalabilidad:** Arquitectura lista para crecer
- **Tiempo real:** Sincronización instantánea
- **SDK multiplataforma:** Web, iOS, Android, React Native
- **Plan gratuito:** Ideal para MVPs y pruebas

Firebase Authentication

Módulo para manejar login, registro y sesión de usuarios.

- Registro con email
- Login con contraseña
- Observador de sesión
- Soporte para Google, Facebook, Anónimo

```
import { getAuth, createUserWithEmailAndPassword } from
'firebase/auth';

const auth = getAuth();
createUserWithEmailAndPassword(auth, email, password);
```

Firestore: Base de datos en tiempo real

Firestore es una base de datos NoSQL, estructurada por colecciones y documentos.

Características:

- Documentos con clave-valor
- Escucha en tiempo real con onSnapshot
- Ideal para tareas, chats, usuarios, contenidos dinámicos

Reglas de seguridad en Firestore

Imprescindible para proteger datos y restringir el acceso:

```
match /tareas/{docId} {
  allow read, write: if request.auth != null;
}
```

¡Es hora de un descansito!

Volvemos en *10'* a seguir *Practicando*



Centro de e-Learning

UTN.BA

Sincronización entre AsyncStorage y Firestore

- Cuando no hay conexión, los datos se guardan localmente.
- Cuando vuelve la red, se sincronizan con Firestore.

Ideal para apps que necesitan modo offline.

```
import NetInfo from '@react-native-community/netinfo';

NetInfo.addEventListener(state => {
  if (state.isConnected) {
    sincronizarPendientes();
  }
});
```

Estrategia general de sincronización

1. Guardar acciones en AsyncStorage
2. Detectar reconexión
3. Subir a Firestore
4. Eliminar lo local
5. Mostrar feedback visual al usuario

¿Qué ofrece AWS como BaaS?

AWS ofrece alternativas a Firebase en su plan gratuito por 12 meses. Puedes utilizar:

- Cognito (autenticación)
- DynamoDB (NoSQL)
- S3 (archivos)
- Lambda (funciones)
- AppSync (GraphQL en tiempo real)

Comparativa Firebase VS AWS

Característica	Firebase	AWS (Amplify / BaaS)
Facilidad de inicio	Muy alta	Media (requiere CLI / IAM)
Escalabilidad	Alta	Muy alta
Autenticación	Integrada	Cognito
Base de datos	Firestore (NoSQL)	DynamoDB (NoSQL)
Almacenamiento	Storage	S3
Sincronización offline	Sí	Sí (con AppSync)

¿Preguntas?
Debatamos un poco los temas de hoy.



Centro de e-Learning
UTN.BA

¡El resto de la clase será práctica!

Centro de e-Learning
UTN.BA

¡Nos vemos la próxima clase!

"Navegación avanzada y transiciones animadas"

Centro de e-Learning

UTN.BA

Bibliografía

- https://docs.expo.dev/versions/latest/sdk/async-storage/
- https://firebase.google.com/docs/auth
- https://firebase.google.com/docs/firestore
- https://aws.amazon.com/free/
- https://docs.amplify.aws/start/q/integration/react-native/